Universidad Rafael Landívar.
Facultad de Ingeniería.
Licenciatura en Ingeniería en Informática y Sistemas.
Manejo e Implementación de Archivos
Docente: Ing. Manuel Gustavo Rojas Vásquez

INFORME TÉCNICO

“Agenda de Contactos con Acceso Aleatorio”

Estudiante: Velásquez González, Luis Manuel
Carné: 1502325

Quetzaltenango, 23 de octubre de 2026

Resumen Ejecutivo

El proyecto implementa una agenda de contactos que almacena la información en un archivo binario de registros de longitud fija llamado contactos.dat. Cada contacto ocupa exactamente **110 bytes** distribuidos en cinco campos: id (10 B), nombre (30 B), telefono (15 B), correo (35 B) y ciudad (20 B). Al no existir separadores ni saltos de línea, la posición de cualquier registro se calcula aritméticamente mediante offset = numero_registro * 110.

El programa abre el archivo en modo binario, ejecuta seek(offset) para posicionar el puntero directamente en el byte deseado, confirma la ubicación con tell() y lee o escribe exactamente 110 bytes. Para editar un contacto se utiliza modificación *in-place* en modo r+b, sobrescribiendo únicamente los 110 bytes correspondientes sin reescribir el resto del archivo. La búsqueda por ID se acelera mediante un índice primario en indice.json con complejidad promedio O(1) frente al costo lineal O(n) de una búsqueda secuencial. Adicionalmente, se resguarda la integridad mediante un hash **SHA-256** (contactos.sha256) actualizado tras cada escritura y un sistema de **respaldo comprimido ZIP**.

1. Arquitectura y Archivos del Sistema

Archivo / Módulo	Tipo	Responsabilidad Principal y Elementos Clave
modelo_contacto.py	Módulo Python	Define TAMANO_REGISTRO = 110, ajustar_campo_utf8(), empaquetar_contacto() y desempaquetar_contacto().
gestor_archivos.py	Módulo Python	Motor de acceso aleatorio (seek/tell), modificación *in-place* y gestión de indice.json.
utilidades_sistema.py	Módulo Python	Cálculo y verificación SHA-256 (hashlib), respaldo ZIP (zipfile) y metadatos (os.stat/os.access).
exportador.py	Módulo Python	Exportación limpia a contactos_exportados.json y generación de reporte_contactos.pdf (fpdf2).
interfaz_brutalista.py	Módulo GUI	Interfaz gráfica en Tkinter, monitor de puntero SEEK/TELL, bitácora y alertas messagebox.
contactos.dat	Binario (Datos)	Almacena los contactos en bloques contiguos de 110 bytes en orden de inserción.
indice.json	JSON UTF-8	Mapa de índice primario {"id": numero_registro} para acceso directo O(1).
contactos.sha256	Texto UTF-8	Almacena el resumen SHA-256 (64 caracteres hexadecimales) de contactos.dat.

2. Estructura del Registro de 110 Bytes (UTF-8)

2.1 Tabla de Estructura y Desplazamientos (Offsets)

Cada registro mide exactamente 110 bytes. Los campos de texto se codifican en UTF-8 y se rellenan con espacios ASCII (0x20, 1 byte) usando .ljust(). Al leer, se decodifican y se limpian con .strip().

Campo	Tipo Lógico	Largo Fijo	Rango de Bytes (Offset)	Ejemplo de Dato	Bytes Datos	Relleno (Espacios)
id	Entero positivo	10 bytes	Bytes 0 a 9	7	1 byte	9 espacios
nombre	Texto UTF-8	30 bytes	Bytes 10 a 39	José Núñez	13 bytes	17 espacios
telefono	Texto UTF-8	15 bytes	Bytes 40 a 54	8888-1234	9 bytes	6 espacios
correo	Texto UTF-8	35 bytes	Bytes 55 a 89	jose.nunez@correo.com	21 bytes	14 espacios
ciudad	Texto UTF-8	20 bytes	Bytes 90 a 109	San José	9 bytes	11 espacios
TOTAL	**Registro Fijo**	**110 bytes**	**Bytes 0 a 109**	**Contacto completo**	**53 bytes**	**57 espacios**

2.2 Ajuste Exacto en Bytes para Caracteres Multibyte (modelo_contacto.py)

En UTF-8, las vocales con tilde (á, é, í, ó, ú) y la ñ ocupan **2 bytes**. Si solo se rellenara contando caracteres, el registro excedería los 110 bytes y desalinearía seek(). La función ajustar_campo_utf8() garantiza la longitud exacta en bytes:n

```
ID_LEN = 10
NOMBRE_LEN = 30
TELEFONO_LEN = 15
CORREO_LEN = 35
CIUDAD_LEN = 20
TAMANO_REGISTRO = 110  # 10 + 30 + 15 + 35 + 20, sin bytes adicionales
```

```
def ajustar_campo_utf8(texto: str, largo_bytes: int) -> bytes:
    if isinstance(largo_bytes, bool) or not isinstance(largo_bytes,
int) or largo_bytes <= 0:
```

```
        raise ValueError("largo_bytes debe ser un entero positivo.")
    if texto is None:
        texto = ""
    elif not isinstance(texto, str):
        texto = str(texto)

    ajustado = texto.ljust(largo_bytes)
    datos = ajustado.encode(CODIFICACION)

    if len(datos) > largo_bytes:
        datos = datos[:largo_bytes]
        datos = datos.decode(CODIFICACION,
errors="ignore").encode(CODIFICACION)

    if len(datos) < largo_bytes:
        datos += BYTE_RELLENO * (largo_bytes - len(datos))

    return datos
```

2.3 Empaquetado y Desempaquetado (`empaquetar_contacto` y `desempaquetar_contacto`)

```
def empaquetar_contacto(id_c, nombre, telefono, correo, ciudad) ->
bytes:

    numero_id = validar_id(id_c)

    piezas = (

        ajustar_campo_utf8(str(numero_id), ID_LEN),
        ajustar_campo_utf8(nombre, NOMBRE_LEN),
        ajustar_campo_utf8(telefono, TELEFONO_LEN),
        ajustar_campo_utf8(correo, CORREO_LEN),
        ajustar_campo_utf8(ciudad, CIUDAD_LEN),

    )

    registro = b"".join(piezas)

    if len(registro) != TAMANO_REGISTRO:

        raise RuntimeError("El registro no mide 110 bytes.")

    return registro
```

Al leer un bloque de 110 bytes, desempaquetar_contacto() extrae las 5 rebanadas según sus longitudes fijas (registro[offset:offset + largo]), decodifica en UTF-8 y elimina el relleno con .strip().

3. Acceso Aleatorio con seek() y tell() y Modificación In-Place

El posicionamiento directo evita recorrer el archivo secuencialmente. El byte inicial de cualquier contacto se determina mediante offset = numero_registro * 110.

3.1 Escritura al Final (escribir_registro_final en gestor_archivos.py)

Se posiciona al final con seek(0, 2), obtiene el offset en bytes mediante tell(), verifica la alineación múltiplo de 110 y calcula numero_registro = posicion_bytes // 110:n

```
with archivo:
archivo.seek(0, 2)                # Posicionar al final del archivo
posicion_bytes = archivo.tell()   # Byte exacto donde inicia el nuevo
registro
if posicion_bytes % TAMANO_REGISTRO != 0:
raise ValueError("El archivo está corrupto: su tamaño no es múltiplo de
110.")
numero_registro = posicion_bytes // TAMANO_REGISTRO
escritos = archivo.write(registro)
archivo.flush()
```

3.2 Lectura Aleatoria Directa (`leer_registro_aleatorio` en `gestor_archivos.py`)

Salta directamente al bloque del contacto mediante `seek(offset)`, confirma la ubicación del cabezal con `tell()` y lee únicamente 110 bytes:

```
offset = numero * TAMANO_REGISTRO
with open(ARCHIVO_CONTACTOS, "rb") as archivo:

    archivo.seek(offset)                # Salto directo al registro
O(1)
```

```
    posicion_actual = archivo.tell()      # Confirmación del byte actual
    registro = archivo.read(TAMANO_REGISTRO)

contacto = desempaquetar_contacto(registro)
contacto["numero_registro"] = numero
contacto["posicion_tell"] = posicion_actual

return contacto
```

3.3 Modificación en el Lugar / In-Place (sobrescribir_registro_en_lugar)

Al editar un contacto, el archivo se abre en modo "r+b" (lectura y escritura binaria sin truncar). Se salta a numero * 110 y se sobrescriben **exclusivamente esos 110 bytes**, manteniendo intactos los registros vecinos y el tamaño total del archivo:

```
offset = numero * TAMANO_REGISTRO
with open(ARCHIVO_CONTACTOS, "r+b") as archivo:

    archivo.seek(0, 2)
    tamano_archivo = archivo.tell()

    if offset + TAMANO_REGISTRO > tamano_archivo:

        raise ValueError("El registro solicitado no existe en
contactos.dat.")

    archivo.seek(offset)                  # Salto directo al registro
por modificar
    posicion_actual = archivo.tell()
    archivo.write(registro)               # Sobrescritura in-place de
110 bytes
    archivo.flush()
```

4. Índice Primario (indice.json) y Complejidad O(1) vs. O(n)

El archivo indice.json asocia cada ID único con su número de registro físico (por ejemplo, {"1": 0, "2": 1, "25": 2}).

- **Búsqueda con Índice Primario O(1):** En memoria, indice.json se gestiona como un diccionario (dict), implementado sobre una **tabla hash**. Consultar el numero_registro de un ID toma tiempo constante promedio **O(1)**, calcular offset = numero_registro * 110 es **O(1)**, y saltar directamente con seek(offset) para leer 110 bytes también es **O(1)**. *(Nota técnica: La localización del registro es O(1); recargar el archivo JSON desde disco añade un costo auxiliar O(m log m) que se vuelve O(1) puro al mantener el índice en caché).*
-
- **Búsqueda Secuencial O(n):** Sin índice, buscar un ID obligaría a recorrer contactos.dat desde el byte 0, leyendo y desempaquetando bloque por bloque hasta n veces, escalando linealmente **O(n)**.

Cantidad de Contactos (n)	**Secuencial: Lecturas (Peor Caso)**	**Secuencial: Bytes Leídos**	**Con Índice: Lecturas**	**Con Índice: Bytes Leídos**	**Factor de Ahorro**
10 contactos	10 lecturas	1,100 B (~1.1 KB)	1 lectura	110 B	10 veces menos
1,000 contactos	1,000 lecturas	110,000 B (~107 KB)	1 lectura	110 B	1,000 veces menos
10,000 contactos	10,000 lecturas	1,100,000 B (~1.05 MB)	1 lectura	110 B	10,000 veces menos
100,000 contactos	100,000 lecturas	11,000,000 B (~10.5 MB)	1 lectura	110 B	100,000 veces menos

5.Integridad SHA-256 y Respaldo Comprimido ZIP

Tras cada alta (agregar_contacto) o edición *in-place* (modificar_por_id), guardar_hash_sha256() lee contactos.dat en bloques de 64 KiB, calcula su huella criptográfica SHA-256 y la guarda en contactos.sha256. Al iniciar la aplicación o desde el menú **Herramientas > Verificar integridad**, verificar_integridad_sha256() compara el hash actual con el almacenado:

```
def _calcular_sha256(ruta: str) -> str:

    sha = hashlib.sha256()

    with open(ruta, "rb") as archivo:
        while True:
            bloque = archivo.read(65536)
            if not bloque:
                break

            sha.update(bloque)
    return sha.hexdigest()
```

Si un usuario altera aunque sea 1 byte de contactos.dat con un editor externo, el efecto avalancha de SHA-256 cambia por completo el resumen hexadecimal y dispara un messagebox.showwarning("ALERTA DE INTEGRIDAD", ...). Por su parte, crear_respaldo_zip() comprime contactos.dat e indice.json mediante zipfile.ZIP_DEFLATED y reporta el tamaño original, tamaño comprimido y porcentaje de compresión: (1 - tamano_zip / tamano_original) * 100.

6. Prueba Manual de Integridad y Evidencias Visuales

Descripción de la prueba: Se registraron dos contactos de prueba (ID 1 e ID 2), generando un archivo contactos.dat de 220 bytes (2 registros × 110 bytes) y el mapeo {"1": 0, "2": 1} en indice.json. Se creó un primer respaldo (respaldo_antes.zip). Luego, se abrió contactos.dat en un editor externo en modo sobrescritura y se modificó un carácter del segundo contacto ("Ana" a "Ani"), manteniendo intacto el tamaño de 220 bytes. Al abrir la aplicación, la verificación automática detectó la discrepancia entre el hash calculado y contactos.sha256, desplegando la ventana **"ALERTA DE INTEGRIDAD"**. Posteriormente se generó un segundo respaldo (respaldo_despues.zip), comprobando la diferencia de hashes y del byte en el offset 120 (0x78). Finalmente, se restauró el respaldo original y el sistema confirmó **"INTEGRIDAD VERIFICADA"**.

6.1 Registro de Capturas de Pantalla

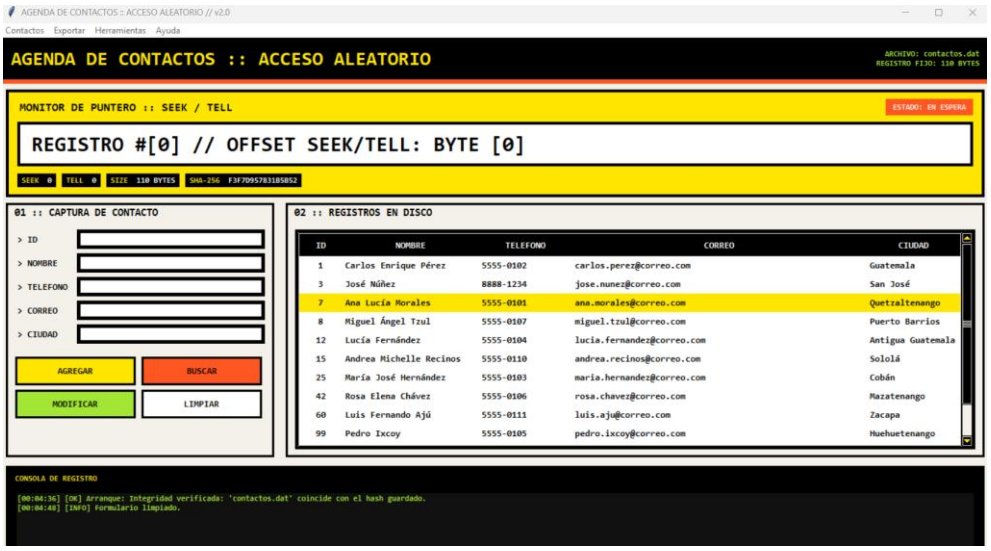


Figura 1. Estado inicial de los archivos (contactos.dat de 220 bytes, indice.json y contactos.sha256) e interfaz principal mostrando el monitor SEEK/TELL.

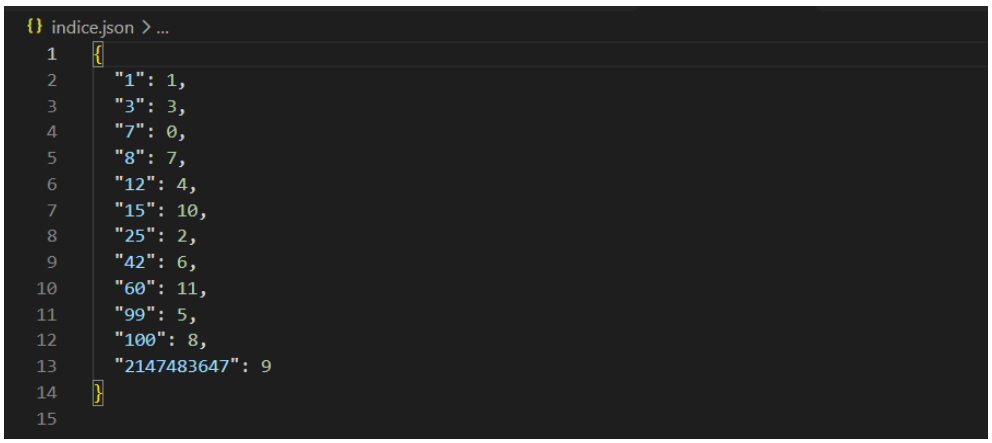


Figura 2. Contenido de indice.json mostrando el mapeo de claves primarias a números de registro ("1": 0, "2": 1).

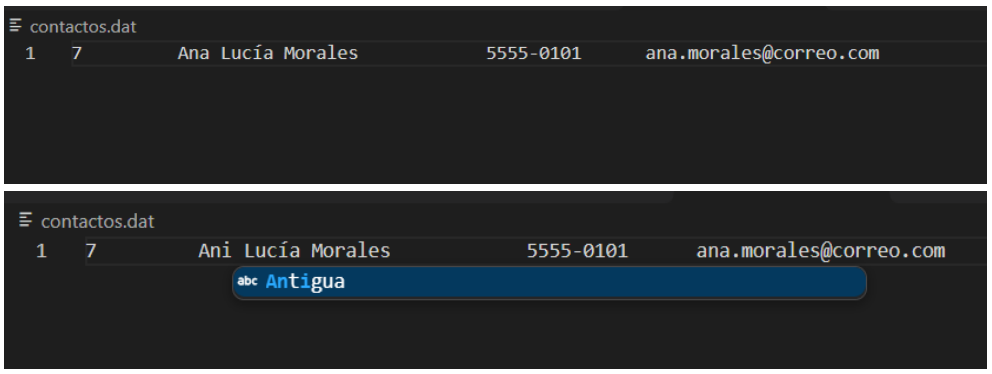


Figura 3. Edición externa de contactos.dat en editor de texto/hexadecimal (cambio de "Ana" por "Ani" conservando 220 bytes).

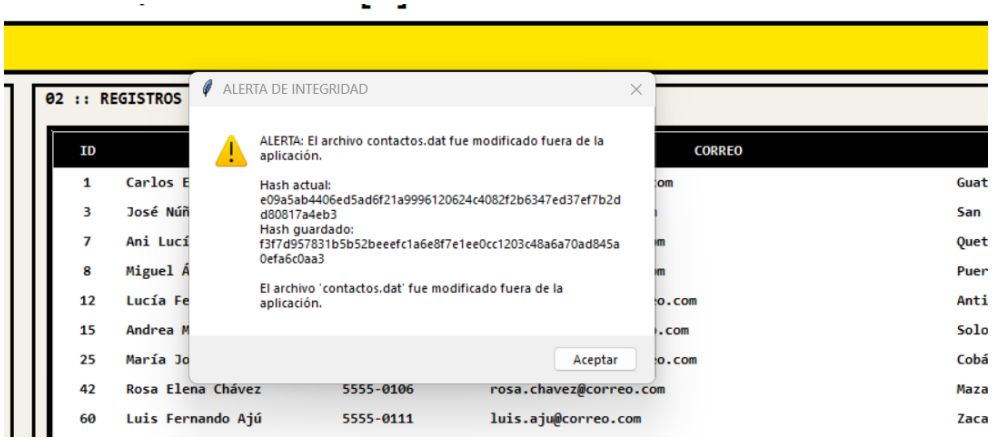
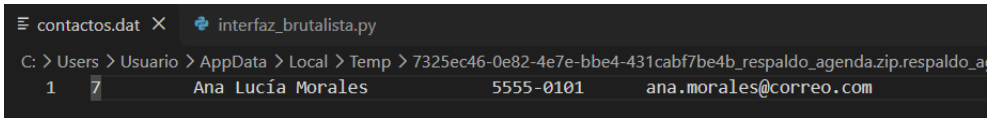


Figura 4. Alerta emergente ALERTA DE INTEGRIDAD mostrada automáticamente al iniciar la aplicación tras la alteración externa, junto con el registro [ALERTA] en la consola.



Archivo	Archivos Originales	Respaldo
Tamaño	142 bytes + 1,320 bytes = 1462 bytes	808 bytes
Formato	json y dat	zip
% de compresión	—	55.26%

Figura 5. Comparativa del Respaldo ZIP (antes y después de la alteración) y cuadro informativo de compresión ZIP.



Figura 6. Diálogo INTEGRIDAD VERIFICADA tras restaurar el archivo original y generación del reporte PDF desde JSON.

6.2 Tabla Comparativa de los Respaldos ZIP (Antes vs. Después)

Momento del Respaldo	Archivo Evaluado	Tamaño (Bytes)	¿Coincide con contactos.sha256?
Antes de la alteración	contactos.dat (ZIP antes)	220 bytes	Sí
Después de la alteración	contactos.dat (ZIP después)	220 bytes	No
Antes / Después	indice.json	[Completar]	Sin cambios